

Convolutional Neural Network

Convolution is the function $(f * g)(x)$ as integral of 2 functions after one function reflected about y axis and shifted

$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau.$$

For discrete summation

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n - m],$$

(1, 2, 3) -> (1, 2, 3)

(4, 5, 6) (6, 5, 4) slide right

1(4) , 1(5) + 2(4) , 1(6) + 2(5) + 3(4) , 2(6) + 3(5) , 3(6) = (4, 13, 28, 27, 18)

Code: `np.convolve((1, 2, 3), (4, 5, 6))`

Used for Moving Averages, Blurring Image Compare with matrix with sum = 1 (Kernel)
Gaussian Blur with Gaussian distribution curve.

FFT Convolve (Fast Fourier Transform):

Original Convolution uses pairwise multiplication resulting in $O(n^2)$ (both same size)

Treat the Input as Coefficients of Polynomial, and evaluate polynomials at those points
(n roots of unity) (reused repeated calc - twiddle factor)

Multiply point wise and use Inverse FFT (IFFT) to convert into polynomial coefficients used for output

Recursive split to even and odd index elements, divide and conquer approach reduces complexity to $O(n \log n)$

CNN:

In CNN convolution is defined as sliding a kernel across image and computing weighted sums

ANN Disadvantages:

High computation, Sensitive to Location

Feature Extraction:

With CNN, the matrix multiplication generates a feature map based on the multiple filter kernel which tolerates small shifts and translations

High activation values (1) indicate learned feature at that location

Early layers learn edges and corners while deep layers learn textures, shapes and parts.

Back propagation is used during training to find the filters and filters don't need to be given but size can be set

Activation Function:

ReLU Function used to bring non linearity (convert negative values to zero), speeds training

Leaky ReLU can be used to prevent neurons dying

Pooling:

The feature map is reduced in size by pooling into a smaller map (eg max of sub matrix)

Reduces dimensions, overfitting, model tolerates distortions

Size: $(m - f + 1) \times (n - f + 1)$ - **Valid Convolution** but corners don't contribute much

To solve this a Padding layer of 0 can be used to add contribution to feature at corner
this generates the dimensions of original - **Same Convolution**
(padding = 'same')

$$\text{Output Size} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

Stride : shift of sub matrix (**strides = (1, 1)**)

Data Augmentation

Generate new rotated or scaled samples for CNN

Classification

After Feature Extraction, feature maps are flattened and passed through connected layers for classification. Soft max is used in the final layer (convert into probability distribution)

Summary:

Input -> Data Augmentation -> Convolution -> ReLU -> Pooling -> (repeat last 3 as needed)
-> Flatten -> Fully Connected Dense Layers -> Softmax -> Classification